

과제1. 정렬 비교 보고서

고급알고리즘 (SIT2001-01) / 이름: ○○○ / 학번: ○○○○○○○○○○○○

GitHub 저장소: <https://github.com/aaron1841-create/algorithm>

1. 비교할 정렬 선택

이번 과제에서는 수업에서 배운 삽입 정렬과 퀵 정렬, 그리고 수업에서 배우지 않은 힙 정렬을 비교했다.

삽입 정렬은 구현이 간단하지만 시간복잡도가 $O(n^2)$ 인 느린 정렬이고, 퀵 정렬은 평균 $O(n \log n)$ 으로 빠르지만 최악의 경우 $O(n^2)$ 이 된다. 힙 정렬은 위키피디아의 정렬 알고리즘 비교 표를 보다가 고르게 되었는데, 퀵 정렬처럼 $O(n \log n)$ 이면서도 최악의 경우에도 $O(n \log n)$ 이 보장된다고 되어 있어서 궁금했다. 그래서 같은 $O(n \log n)$ 인 퀵 정렬과 힙 정렬이 실제로 얼마나 차이가 나는지, 그리고 $O(n^2)$ 인 삽입 정렬과는 얼마나 차이가 나는지 확인해 보고 싶었다.

또 샘플 보고서에서는 비교한 세 정렬이 모두 안정 정렬이었는데, 퀵 정렬과 힙 정렬은 불안정 정렬이라서 안정성 차이도 함께 볼 수 있을 것이라고 생각했다.

2. 코드 구성

코드는 교수님이 올려주신 샘플 저장소의 구조를 그대로 사용했다. 샘플에서는 정렬 함수들을 함수 포인터로 하나의 표 (SORT_ALGORITHMS)에 넣어 두고, 측정하는 코드와 테스트 코드는 이 표만 보고 정렬을 호출한다. 그래서 정렬을 바꾸고 싶으면 파일을 추가하고 표에 한 줄만 넣으면 된다. 나는 샘플에 있던 버블 정렬과 블록 정렬 파일을 지우고 quickSort.c와 heapSort.c를 새로 만들어서 표에 넣었다. 삽입 정렬은 샘플 코드를 그대로 썼다.

표에는 각 정렬이 안정 정렬인지도 적게 되어 있는데, 삽입 정렬은 안정, 퀵 정렬과 힙 정렬은 불안정으로 적었다. 테스트 코드가 실제로 정렬을 돌려서 이 값이 맞는지 확인한다.

테스트는 make test로 실행하고, 섞인 배열, 이미 정렬된 배열, 역순 배열, 같은 값이 있는 배열, 원소가 1개인 배열, 빈 배열을 넣어 보고 표준 라이브러리의 qsort 결과와도 비교한다. 원래 샘플 테스트는 크기 0부터 200까지의 배열에서 안정성까지 검사했는데, 퀵 정렬과 힙 정렬은 원래 불안정하기 때문에 안정 정렬로 적힌 것만 안정성을 검사하도록 조건을 바꿨다. 퀵 정렬은 피벗을 고르는 방법 두 가지에 대해서도 테스트를 추가했고, 최종적으로 51개의 검사가 모두 통과했다.

3. 알고리즘 설명

3.1 삽입 정렬

삽입 정렬은 배열 앞부분을 정렬된 상태로 두고, 뒤에서 원소를 하나씩 꺼내서 앞쪽의 알맞은 자리에 끼워 넣는 방법이다. 자기보다 큰 원소들을 한 칸씩 뒤로 밀고 빈 자리에 넣는다. 같은 값을 만나면 더 앞으로 가지 않고 멈추기 때문에 안정 정렬이다. 이미 정렬된 배열이 들어오면 각 원소가 바로 앞 원소와 한 번씩만 비교하고 끝나기 때문에 매우 빠르다.

3.2 퀵 정렬

퀵 정렬은 피벗을 하나 정한 다음, 피벗보다 작은 값은 왼쪽으로, 큰 값은 오른쪽으로 나누고(분할), 나눠진 양쪽을 다시 퀵 정렬로 정렬하는 방법이다.

분할은 배열의 양쪽 끝에서 가운데로 오면서 진행했다. 왼쪽에서는 피벗보다 크거나 같은 값을 찾고, 오른쪽에서는 피벗보다 작거나 같은 값을 찾아서 둘을 서로 바꾼다. 두 위치가 엇갈리면 멈추고 피벗을 그 자리로 옮긴다. 피벗과 같은 값에서도 멈추게 했는데, 이렇게 하면 같은 값이 많은 배열에서도 한쪽으로 몰리지 않고 비교적 반씩 나뉜다. 대신 멀리 떨어진 원소끼리 자리를 바꾸기 때문에 같은 값의 순서가 바뀔 수 있어서 불안정 정렬이다.

피벗을 맨 앞 원소로 고르면 이미 정렬된 배열에서는 매번 한쪽에 원소가 하나도 없게 나뉘어서 $O(n^2)$ 이 된다고 배웠다. 그래

서 기본 설정은 맨 앞, 가운데, 맨 뒤 세 값의 중앙값을 피벗으로 쓰도록 했고, 두 방법의 차이는 실험에서 비교했다.

처음에 중앙값 피벗을 만들 때는 세 값 중 중앙값이 있는 위치를 찾아서 그 원소를 맨 앞 원소와 바꾸기만 했다. 그런데 이렇게 하니 정렬된 배열은 괜찮았는데 역순 배열에서 재귀 깊이가 1,002까지 가고 비교 횟수도 200만 번이 넘게 나왔다. 원인을 찾아보니 맨 앞에 있던 가장 큰 값이 교환 때문에 배열 가운데로 옮겨지면서 그 뒤로 분할이 계속 한쪽으로 치우치는 것 같았다. 그래서 세 값을 먼저 크기 순서대로 정리한 다음 가운데 값을 피벗으로 쓰도록 고쳤고, 그 후에는 재귀 깊이가 12로 정상적으로 나왔다. 중앙값 피벗도 구현을 조금 잘못하면 특정 입력에서 느려질 수 있다는 것을 알게 되었다.

3.3 힙 정렬 (수업에서 배우지 않은 정렬)

힙 정렬은 수업에서 배우지 않아서 AI(Claude)에게 질문하면서 공부했다. 질문한 내용과 이해한 내용을 정리하면 다음과 같다.

먼저 힙이 무엇인지 물어봤다. 힙은 완전 이진 트리인데, 최대 힙은 모든 부모 노드가 자식 노드보다 크거나 같은 트리라고 한다. 그래서 가장 큰 값은 항상 맨 위(루트)에 있다. 완전 이진 트리가기 때문에 포인터를 쓰지 않고 배열에 위에서부터 한 층씩 차례대로 저장할 수 있고, 루트가 0번일 때 i 번 노드의 자식은 $2i+1$ 번과 $2i+2$ 번이 된다. 예를 들어 배열 $[10, 5, 3, 4, 1]$ 은 10이 루트이고 그 자식이 5와 3, 5의 자식이 4와 1인 최대 힙이다.

다음으로 힙을 이용해서 어떻게 정렬하는지 물어봤다. 힙 정렬은 두 단계로 되어 있다. 첫 번째 단계는 배열을 최대 힙으로 만드는 것이다. 자식이 있는 마지막 노드부터 루트까지 거꾸로 가면서, 각 노드를 자식과 비교하고 자식이 더 크면 아래로 내려보낸다. 두 번째 단계는 루트에 있는 최댓값을 배열 맨 끝으로 보내고, 힙의 크기를 하나 줄인 다음, 새로 루트에 온 값을 다시 아래로 내려보내서 힙을 고치는 것이다. 이것을 반복하면 배열 뒤쪽부터 큰 값이 차례대로 쌓이면서 오름차순으로 정렬된다.

이해가 잘 안 돼서 $[4, 10, 3, 5, 1]$ 을 직접 손으로 정렬해 보았다. 힙을 만들 때 1번 노드인 10은 자식 5와 1보다 크니까 그대로 두고, 0번 노드인 4는 자식 중 큰 값인 10과 자리를 바꾸고, 다시 내려간 자리에서 자식 5와 자리를 바꿔서 $[10, 5, 3, 4, 1]$ 이 된다. 그 다음 10을 맨 뒤로 보내고 남은 부분을 다시 힙으로 고치면 $[5, 4, 3, 1, 10]$ 이 되고, 같은 과정을 반복하면 $[4, 1, 3, 5, 10]$, $[3, 1, 4, 5, 10]$, $[1, 3, 4, 5, 10]$ 순서로 정렬이 끝났다.

세 번째로, 첫 번째 단계인 힙 만들기의 시간복잡도를 물어봤다. 나는 원소 $n/2$ 개를 각각 최대 $\log n$ 만큼 내려보내니까 $O(n \log n)$ 이라고 생각했는데, AI는 트리의 아래쪽에 있는 노드가 대부분이고 이 노드들은 내려갈 거리가 거의 없어서 전부 더하면 $O(n)$ 이 된다고 설명했다. 잘 와닿지 않아서 힙 만들기 부분만 파이썬으로 짜서 비교 횟수를 세어 보았다

(tools/heap_build_count.py). n 이 1,000일 때 1,898번, 32,000일 때 60,128번이었고, 비교 횟수를 n 으로 나눈 값이 n 이 커져도 계속 약 1.9로 거의 같았다. $n \log n$ 이었다면 이 값이 점점 커져야 하므로 힙 만들기는 정말 $O(n)$ 이라는 것을 확인할 수 있었다. 물론 두 번째 단계가 $O(n \log n)$ 이라서 힙 정렬 전체는 $O(n \log n)$ 이다.

네 번째로 퀵 정렬과 비교하면 어떤 장단점이 있는지 물어봤다. 힙 정렬은 입력이 어떻게 들어와도 $O(n \log n)$ 이 보장되고, 재귀도 쓰지 않아서 추가 메모리가 거의 없다는 것이 장점이다. 반면 실제 속도는 보통 퀵 정렬이 더 빠르다고 한다. 힙 정렬은 i 번과 $2i+1$ 번처럼 멀리 떨어진 위치를 왔다 갔다 하면서 비교하고, 비교 횟수 자체도 퀵 정렬보다 많기 때문이라고 했다. 그리고 멀리 떨어진 원소끼리 자리를 바꾸기 때문에 불안정 정렬이다.

마지막으로 힙 정렬이 실제로 어디에 쓰이는지 물어봤는데, C++의 `std::sort`가 사용하는 인트로 정렬(introsort)에 들어간다고 한다. 인트로 정렬은 처음에는 퀵 정렬로 정렬하다가 재귀가 너무 깊어지면 힙 정렬로 바꾸고, 정렬할 구간이 작아지면 삽입 정렬로 마무리하는 방식이라고 한다. 이번 과제에서 비교한 세 정렬이 모두 들어가 있어서 신기했다.

힙 정렬을 구현할 때는 원소를 아래로 내려보내는 부분을 교환으로 하지 않았다. 교환을 하면 한 번에 값 복사가 3번 일어나기 때문에, 내려보낼 값을 임시 변수에 저장해 두고 큰 자식을 한 칸씩 위로 올린 다음 마지막에 저장해 둔 값을 넣는 방식으로 했다. 삽입 정렬에서 원소를 한 칸씩 뒤로 미는 것과 비슷한 방법이다. 재귀 없이 반복문으로만 구현했다.

4. 실험

4.1 실험 방법

각 원소는 정렬 기준인 key와 입력 순서를 저장한 tag로 이루어져 있고, key로만 정렬했다. 정렬이 끝난 후 key가 같은 원소들의 tag가 원래 순서대로 남아 있으면 안정 정렬로 판단했다. 입력은 무작위 배열, 이미 정렬된 배열, 역순 배열, 같은 값이 많은 배열(key가 0부터 7까지) 네 가지를 사용했고, 난수 시드를 고정해서 매번 같은 입력이 들어가게 했다.

시간은 clock() 함수로 정렬하는 부분만 측정해서 3번 평균을 냈다. 그런데 시간은 컴퓨터 상태에 따라 조금씩 달라지기 때문에 비교 횟수와 이동 횟수도 같이 기록했다. 비교 횟수와 이동 횟수는 몇 번을 실행해도 똑같이 나온다. 전체 측정 결과는 저장소의 report/results.csv와 report/pivot.csv 파일에 있다.

4.2 입력 종류에 따른 비교 (n = 4,000)

무작위 배열에서는 삽입 정렬이 약 34.6ms, 퀵 정렬이 약 0.59ms, 힙 정렬이 약 0.90ms 걸렸다. 비교 횟수는 삽입 정렬이 약 395만 번, 퀵 정렬이 약 5.6만 번, 힙 정렬이 약 8.3만 번이었다. 삽입 정렬이 퀵 정렬보다 50배 넘게 느렸다.

이미 정렬된 배열에서는 결과가 반대로 나왔다. 삽입 정렬은 0.019ms로 세 정렬 중 가장 빨랐고, 비교 횟수도 3,999번(n-1번) 밖에 되지 않았다. 퀵 정렬은 0.24ms, 힙 정렬은 0.73ms였다.

역순 배열은 삽입 정렬의 최악의 경우라서 약 69.6ms가 걸렸고, 비교 횟수가 약 800만 번으로 이론값 $n^2/2 = 8,000,000$ 과 거의 같았다. 퀵 정렬은 0.42ms, 힙 정렬은 0.84ms였다.

같은 값이 많은 배열에서는 삽입 정렬 30.5ms, 퀵 정렬 0.55ms, 힙 정렬 0.75ms였다.

여기서 알 수 있었던 점은 다음과 같다. 첫째, 삽입 정렬은 정렬된 입력에서만 빠르고 나머지 입력에서는 퀵 정렬과 힙 정렬보다 수십 배에서 백 배 이상 느렸다. 둘째, 힙 정렬은 네 가지 입력 모두에서 0.73~0.90ms 사이로 걸린 시간이 거의 비슷했다. 정렬된 입력이 들어와도 힙을 만드는 과정에서 순서가 다 섞이기 때문에 더 빨라지지 않는 것 같다. 셋째, 퀵 정렬은 모든 입력에서 힙 정렬보다 빨랐다.

안정성은 같은 값이 많은 배열에서 확인할 수 있었다. 이 입력에서 삽입 정렬은 안정적으로 정렬되었지만 퀵 정렬과 힙 정렬은 같은 값의 순서가 바뀌었다. 재미있었던 점은 무작위 배열에서는 퀵 정렬과 힙 정렬도 안정적인 것으로 나왔다는 것이다. 무작위 배열은 rand() 값이라 같은 key가 거의 없어서 순서가 바뀔 일이 없었기 때문이다. 안정성은 같은 값이 많은 입력으로 확인해야 한다는 것을 알게 되었다.

4.3 배열 크기에 따른 비교 (무작위 입력)

n을 1,000부터 32,000까지 두 배씩 늘리면서 측정했다. n이 두 배가 될 때마다 삽입 정렬의 비교 횟수는 약 4배씩 늘어났고, 퀵 정렬과 힙 정렬은 약 2.2배씩 늘어났다. 삽입 정렬이 $O(n^2)$, 나머지 둘이 $O(n \log n)$ 이라는 것과 잘 맞는 결과이다.

n이 32,000일 때 삽입 정렬은 약 2.2초가 걸렸고, 퀵 정렬은 5.7ms, 힙 정렬은 8.4ms가 걸렸다. 삽입 정렬과 퀵 정렬은 약 390배 차이가 났다. n이 작을 때는 차이가 20배 정도였는데 n이 커질수록 차이가 훨씬 커졌다.

퀵 정렬과 힙 정렬은 둘 다 $O(n \log n)$ 이지만 힙 정렬이 비교를 약 1.5배 더 많이 했고, 시간도 약 1.4~1.6배 더 걸렸다. 힙 정렬은 원소를 한 층 내려보낼 때마다 두 자식을 비교하고, 큰 자식과 내려보내는 값을 또 비교해서 한 층에 비교를 2번씩 하기 때문인 것 같다. AI가 설명해 준 것처럼 이론상 복잡도가 같아도 실제 속도는 다를 수 있다는 것을 확인했다.

4.4 퀵 정렬의 피벗 선택 실험 (n = 4,000)

퀵 정렬에서 피벗을 맨 앞 원소로 고를 때와 세 값의 중앙값으로 고를 때를 비교했다.

무작위 배열에서는 두 방법이 거의 비슷했다. 맨 앞 피벗이 0.58ms, 중앙값 피벗이 0.63ms로 오히려 중앙값 피벗이 조금 느렸는데, 중앙값을 구하는 데 비교와 교환이 추가로 들기 때문인 것 같다.

하지만 정렬된 배열과 역순 배열에서는 큰 차이가 났다. 맨 앞 피벗은 정렬된 배열에서 약 25.4ms가 걸리고 비교를 약 800만 번 했는데, 이것은 삽입 정렬의 최악의 경우와 비슷한 수준이다. 재귀 깊이도 4,000으로 n과 같았다. 매번 한쪽에 원소가 하나도 없게 나뉘었기 때문이다. 반면 중앙값 피벗은 같은 입력에서 0.23ms, 재귀 깊이 12였다. $\log_2 4000$ 이 약 12이므로 매번 거의 정확히 반으로 나뉜 것이다. 역순 배열에서도 결과가 거의 같았다.

이미 정렬되어 있거나 거의 정렬된 데이터는 실제로도 자주 있을 것 같기 때문에 맨 앞 원소를 피벗으로 쓰면 위험하다는 것을 알 수 있었다. 그리고 앞에서 말한 것처럼 중앙값 피벗도 잘못 구현하면 느려지는 경우가 있었기 때문에, 최악의 경우를 확실하게 막으려면 힙 정렬처럼 보장된 방법이 필요하다고 생각했다.

4.5 메모리

세 정렬 모두 배열 안에서 정렬하는 제자리 정렬이라서 추가로 사용하는 메모리는 원소 하나를 임시로 저장하는 8바이트뿐이었다. 다만 퀵 정렬은 재귀를 사용하기 때문에 재귀 깊이만큼 스택 메모리를 더 쓴다. n 이 32,000일 때 퀵 정렬의 재귀 깊이는 27이었고, 삽입 정렬과 힙 정렬은 재귀를 쓰지 않아서 1이었다. 그래서 세 정렬 중에서 추가 메모리가 정말로 $O(1)$ 이면서 $O(n \log n)$ 인 정렬은 힙 정렬뿐이다.

5. 결론

실험 결과를 정리하면, 평균적인 무작위 데이터에서는 퀵 정렬이 가장 빨랐고, 이미 정렬된 데이터에서는 삽입 정렬이 가장 빨랐다. 힙 정렬은 퀵 정렬보다 조금 느렸지만 어떤 입력이 들어와도 걸리는 시간이 거의 같았다. 같은 값의 순서를 유지해야 하는 경우에는 세 정렬 중 삽입 정렬만 사용할 수 있다.

과제를 하기 전에는 시간복잡도가 가장 좋은 정렬이 제일 좋은 정렬이라고 생각했는데, 실험을 해 보니 입력이 어떤 모양인지에 따라 좋은 정렬이 달라진다는 것을 알게 되었다. 퀵 정렬은 평균적으로 가장 빨랐지만 피벗을 잘못 고르면 삽입 정렬만큼 느려졌고, 삽입 정렬은 느리지만 정렬된 입력에서는 가장 빨랐다.

힙 정렬을 공부하면서 알게 된 인트로 정렬도 이제는 왜 그렇게 만들었는지 이해가 된다. 평소에는 빠른 퀵 정렬을 쓰다가, 피벗 실험에서 본 것처럼 재귀가 너무 깊어지면 힙 정렬로 바뀌어서 최악의 경우를 막고, 작은 구간은 삽입 정렬로 처리하는 것이다. 이번에 비교한 세 정렬의 장점만 모은 방법이라고 생각한다.

이번 실험의 한계도 있다. 시간은 한 컴퓨터에서만 측정했기 때문에 다른 환경에서는 달라질 수 있다. 그리고 n 이 커지면 삽입 정렬이 너무 오래 걸려서 32,000까지만 측정했다. 퀵 정렬도 작은 구간에서 삽입 정렬로 바꾸는 것 같은 최적화를 하지 않은 기본적인 형태로 구현했다.

6. AI 활용

힙 정렬의 개념은 AI(Claude)에게 질문하면서 공부했고, 샘플 저장소의 구조를 이해하고 코드를 작성할 때도 도움을 받았다. AI가 설명한 내용 중에서 힙 만들기가 $O(n)$ 이라는 것, 힙 정렬이 퀵 정렬보다 비교를 더 많이 한다는 것, 맨 앞 피벗을 쓰면 정렬된 입력에서 $O(n^2)$ 이 된다는 것은 직접 실험해서 비교 횟수와 재귀 깊이로 확인했다.